

Telehealth Platform

Integration Guide

PracticeQ · QuickBooks Online · Life File Pharmacy · Spruce SMS

■■ Current state — all integrations are mocked. They write structured data to localStorage and log every event, but make no real API calls. This document explains what each integration does today, what production requires, and exactly where to add the code.

Confidential · Demo Use Only · Not for Clinical Use

Overview — Order Lifecycle

Every patient order passes through four integration touchpoints. Each is triggered automatically as the order advances through statuses. All events are written to the Integration Logs table (Admin → Integration Logs).

Step	Trigger	Integration	Data sent
1	Patient completes intake	PracticeQ	Patient info, questionnaire, consent, uploads
2	Payment captured	QuickBooks	Customer record + invoice + payment

3	Provider approves order	Life File	Rx order with NPI, drug, dose, qty, shipping
4	Each status change	Spruce SMS	Templated SMS to patient phone

1. PracticeQ

Purpose: Electronic health record (EHR) / practice management. Receives the full patient intake packet so the provider can review it inside PracticeQ as well as the custom provider dashboard.

What the mock does today

Builds a `PracticeQPacket` object and stores it in `localStorage`:

- Patient demographics (name, DOB, address, phone, email)
- All questionnaire answers
- Signed consent record (telehealth, pharmacy, payment, privacy)
- Uploaded files — driver's license and selfie video (stored as base64)
- Product name and dose label requested

Sets `order.practiceQStatus = "completed"` and writes a success entry to the Integration Logs.

What production needs

Item	Detail
API endpoint	POST https://app.practicq.com/api/v1/intakes (confirm URL with PracticeQ)
Authentication	Bearer token in Authorization header
Env variable	PRACTICEQ_API_KEY
File handling	Upload files to PracticeQ document endpoint first; reference returned IDs in packet
Error handling	On failure: set <code>practiceQStatus = "error"</code> , write error log, retry with backoff

Where to add the production code

File to create: `lib/integrations/practicq.ts`

```
// lib/integrations/practicq.ts
export async function submitIntakePacket(orderId: string): Promise<void> {
  const packet = db.practicqDb.getByOrder(orderId);
  const patient = db.patientDb.getById(packet.patientId);
```

```
// 1. Upload documents first
const docIds = await uploadDocuments(packet.packetData.uploads);

// 2. Submit intake packet
const res = await fetch(process.env.PRACTICEQ_BASE_URL + "/intakes", {
  method: "POST",
  headers: { Authorization: "Bearer " + process.env.PRACTICEQ_API_KEY },
  body: JSON.stringify({ ...packet.packetData, documentIds: docIds }),
});

if (!res.ok) {
  db.orderDb.update(orderId, { practiceQStatus: "error" });
  db.integrationLogDb.create({ integrationName: "practiceq",
    action: "Intake packet failed", status: "error", ... });
  throw new Error(await res.text());
}

db.orderDb.update(orderId, { practiceQStatus: "completed" });
db.integrationLogDb.create({ integrationName: "practiceq",
  action: "Intake packet submitted", status: "success", ... });
}
```

Trigger point: Call `submitIntakePacket(order.id)` at the end of `app/start/payment/page.tsx`, after payment is confirmed.

2. QuickBooks Online

Purpose: Accounting. Creates a customer record and invoice for every completed payment so revenue appears automatically in QuickBooks without manual entry.

What the mock does today

- Creates a `QuickBooksRecord` with `customerRefId`, `invoiceId`, `invoiceNumber`, `amount`, `taxAmount`
- Sets `status = "created"` and stores in `localStorage`
- Writes a success entry to `Integration Logs`

What production needs

Item	Detail
Auth method	OAuth 2.0 — use <code>intuit-oauth</code> or <code>node-quickbooks</code> SDK
Env variables	<code>QUICKBOOKS_CLIENT_ID</code> , <code>QUICKBOOKS_CLIENT_SECRET</code> , <code>QUICKBOOKS_REALM_ID</code> , <code>QUICKBOOKS_REFRESH_TOKEN</code>
Customer	<code>GET /v3/company/{realmId}/query</code> to find existing customer, create if not found

Invoice	POST /v3/company/{realmId}/invoice
Payment	POST /v3/company/{realmId}/payment — links payment to invoice
Token refresh	Access tokens expire after 1 hour — implement automatic refresh via refresh token

Where to add the production code

File to create: lib/integrations/quickbooks.ts

```
// lib/integrations/quickbooks.ts
export async function createInvoice(
  orderId: string, paymentId: string
): Promise<void> {
  const payment = db.paymentDb.getById(paymentId);
  const order = db.orderDb.getById(orderId);
  const patient = db.patientDb.getById(order.patientId);

  const token = await getAccessToken(); // handle OAuth refresh
  const baseUrl = `https://quickbooks.api.intuit.com/v3/company/${process.env.QUICKBOOKS_REALM_ID}`;

  // 1. Find or create QBO customer
  const customerId = await findOrCreateCustomer(patient, token, baseUrl);

  // 2. Create invoice
  const invoice = await createQBOInvoice(order, customerId, token, baseUrl);

  // 3. Apply payment
  await applyPayment(invoice.Id, payment.amount, token, baseUrl);

  db.quickbooksDb.update(orderId, { status: "invoiced" });
  db.integrationLogDb.create({ integrationName: "quickbooks",
    action: "Invoice created & paid", status: "success", ... });
}
```

Trigger point: Call `createInvoice(order.id, payment.id)` inside the payment processing step in `app/start/payment/page.tsx`, immediately after the payment is marked completed.

3. Life File (Pharmacy)

Purpose: Compounding pharmacy partner. Receives the Rx order containing the prescriber NPI, drug, strength, quantity, and patient shipping address, then fulfills and ships the medication.

What the mock does today

- Builds a PharmacyOrder payload with prescriber block, practice block, patient demographics, shipping address, and rxs[] array
- Generates a fake Life File order ID (LF_XXXXXX) and fake UPS tracking number

- Stores the full payload in localStorage
- Writes logs at submission and each status update (processing, fulfilled, shipped, delivered)

What production needs

Item	Detail
API credentials	LIFE_FILE_API_KEY, LIFE_FILE_BASE_URL
Prescriber NPI	Replace placeholder 1234567890 — store as PRESCRIBER_NPI, PRESCRIBER_NAME, PRESCRIBER_PHONE
Practice NPI	Replace placeholder 0987654321 — store as PRACTICE_NPI, PRACTICE_NAME, PRACTICE_PHONE
Status updates	Implement webhook at /api/webhooks/lifefile or poll Life File API
Tracking numbers	Real tracking number provided by Life File once shipped — trigger Spruce SMS immediately
Error handling	Pharmacy rejections must alert provider; implement retry logic

Where to add the production code

Files to create:

- lib/integrations/lifefile.ts — submit order to Life File API
- app/api/webhooks/lifefile/route.ts — receive status updates & tracking numbers

```
// lib/integrations/lifefile.ts
export async function submitPharmacyOrder(orderId: string): Promise<void> {
  const pharmacyOrder = db.pharmacyOrderDb.getByOrder(orderId);

  const res = await fetch(process.env.LIFE_FILE_BASE_URL + "/orders", {
    method: "POST",
    headers: { "X-API-Key": process.env.LIFE_FILE_API_KEY! },
    body: JSON.stringify(pharmacyOrder.payload),
  });

  if (!res.ok) throw new Error(await res.text());

  const { orderId: lifeFileOrderId } = await res.json();
  db.pharmacyOrderDb.update(pharmacyOrder.id, {
    lifeFileOrderId, status: "submitted", submittedAt: new Date().toISOString(),
  });
  db.orderDb.update(orderId, { pharmacyStatus: "submitted" });
  db.integrationLogDb.create({ integrationName: "lifefile",
    action: "Pharmacy order submitted", status: "success",
    details: { lifeFileOrderId }, ... });
}
```

```
// app/api/webhooks/lifefile/route.ts
export async function POST(req: Request) {
  const { orderId, status, trackingNumber } = await req.json();
  db.pharmacyOrderDb.update(..., { status, trackingNumber });
  db.orderDb.update(orderId, { pharmacyStatus: status });
  if (trackingNumber) {
    await sendSMS(patientId, orderId, "tracking", { trackingNumber });
  }
  return Response.json({ ok: true });
}
```

Trigger point: Call `submitPharmacyOrder(order.id)` when a provider approves an order — currently in `app/provider/patients/[id]/page.tsx`.

4. Spruce SMS

Purpose: Patient communication. Sends an SMS at every key order milestone so patients always know where their order is without needing to log in.

Message templates (all editable via Admin → CMS)

Template key	Trigger	Message summary
intake_received	Patient submits intake	We've received your intake. A provider will review shortly.
payment_received	Payment captured	We've received your payment for \$X. Order ID: ...
sent_to_pharmacy	Order sent to Life File	Your order has been sent to our pharmacy partner.
fulfilled	Pharmacy fulfills order	Your order has been fulfilled. Tracking info coming soon.
tracking	Order shipped	Your tracking info: UPS123456789. Contact us with questions.

What production needs

Item	Detail
API credentials	SPRUCE_API_KEY, SPRUCE_BASE_URL, SPRUCE_FROM_NUMBER
Endpoint	POST /api/v1/messages with to, from, body fields

Template vars	Interpolate {{patientName}}, {{trackingNumber}}, {{amount}} at send time
Opt-out handling	Honour STOP replies — Spruce handles this automatically
Delivery receipts	Use Spruce webhooks to update SpruceMessage.status to sent/failed

Where to add the production code

File to create: lib/integrations/spruce.ts

```
// lib/integrations/spruce.ts
export async function sendSMS(
  patientId: string,
  orderId: string,
  templateKey: string,
  variables: Record<string, string> = {}
): Promise<void> {
  const template = db.messageTemplateDb.getByKey(templateKey);
  const patient = db.patientDb.getById(patientId);

  // Interpolate variables: {{patientName}} → "Alice"
  let body = template.body;
  Object.entries({ patientName: patient.firstName, ...variables })
    .forEach(([k, v]) => { body = body.replaceAll(`{{${k}}}`, v); });

  const res = await fetch(process.env.SPRUCE_BASE_URL + "/messages", {
    method: "POST",
    headers: { Authorization: "Bearer " + process.env.SPRUCE_API_KEY },
    body: JSON.stringify({ to: patient.phone,
      from: process.env.SPRUCE_FROM_NUMBER, body }),
  });

  const msg = db.spruceDb.create({ ..., status: res.ok ? "sent" : "failed" });
  db.integrationLogDb.create({ integrationName: "spruce",
    action: `SMS sent: ${templateKey}`, status: res.ok ? "success" : "error", ... });
}
```

Environment Variables

Add these to `.env.local` for development and to your hosting platform (Vercel / AWS / etc.) for production.

```
# PracticeQ
PRACTICEQ_API_KEY=
PRACTICEQ_BASE_URL=https://app.practicq.com/api/v1

# QuickBooks Online
QUICKBOOKS_CLIENT_ID=
QUICKBOOKS_CLIENT_SECRET=
QUICKBOOKS_REALM_ID=
QUICKBOOKS_REFRESH_TOKEN=
```

```
# Life File Pharmacy
LIFE_FILE_API_KEY=
LIFE_FILE_BASE_URL=
PRESCRIBER_NPI=
PRESCRIBER_NAME=
PRESCRIBER_PHONE=
PRACTICE_NPI=
PRACTICE_NAME=
PRACTICE_PHONE=

# Spruce SMS
SPRUCE_API_KEY=
SPRUCE_BASE_URL=
SPRUCE_FROM_NUMBER=
```

Production Go-Live Checklist

PracticeQ

- Create lib/integrations/practiceq.ts
- Obtain API key from PracticeQ account
- Map packet schema to PracticeQ intake format
- Test file upload in PracticeQ sandbox
- Wire call into app/start/payment/page.tsx

QuickBooks

- Create lib/integrations/quickbooks.ts
- Complete OAuth 2.0 setup and store refresh token
- Test customer + invoice creation in QBO sandbox
- Wire call into payment page after payment.status = completed
- Set up token refresh cron job

Life File

- Create lib/integrations/lifefile.ts
- Obtain real PRESCRIBER_NPI and PRACTICE_NPI from your licensed provider
- Create app/api/webhooks/lifefile/route.ts
- Register webhook URL with Life File
- Test order submission in Life File sandbox
- Wire call into provider approval in app/provider/patients/[id]/page.tsx

Spruce SMS

- Create lib/integrations/spruce.ts

- Obtain Spruce API key and from-number
- Review and update SMS templates in Admin → CMS
- Replace all mock `spruceDb.create()` calls with `sendSMS()`
- Test delivery receipts via Spruce webhook

General

- Populate all env variables on hosting platform
- Run end-to-end test order through every status
- Verify Integration Logs capture all events correctly
- Set up error alerting (e.g. email/Slack on integration errors)